# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 2 –DAST SIMULATION

| | | | |
|---|---|---|---|
| **Course Code:** | CSA5803 | **Course Title:** | DEVSECOPS ENGINEERING AND APPLICATION SECURITY |
| **CO Assessed:** | CO3 – Precision (BL1, BL2, BL3) | **Assessment:** | Assessment Tool 2 |
| **Weightage:** | 30% of CIA | **Total Marks:** | 100 |
| **CO3 Statement:** | Simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA). | | |
| **Student Name:** | M.GOKUL | **Reg. No.:** | 192565067 |
| **Section / Batch:** | 2025 | **Date:** | 13/08/2026 |

**Instructions to Students**

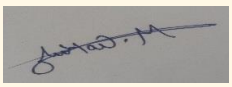
- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

| Question Type | Focus Area | Questions | Marks |
|---|---|---|---|
| **DAST SIMULATION** | Application based | Q1 | Q1 –100 |
| | **GRAND TOTAL:** | **100 Marks** | |

**Code of Conduct**

*I **M.GOKUL 192565067**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:



**DAST SIMULATION — COMPLETE PROJECT**

## 1. Project Title

**Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA) for a Web Application**

---

## 2. Abstract

DAST is a security testing technique used to identify vulnerabilities in a running web application. Unlike static testing, DAST examines the application from an external perspective by sending requests and analyzing responses.
In this project, a simple Flask web application is developed and deployed locally. OWASP ZAP is then used to perform dynamic security testing on the running application. The scan identifies possible security issues and generates security findings.
Software Composition Analysis (SCA) is also included to identify security risks in third-party software dependencies used by the application. The results from DAST and SCA are analyzed, documented, and used to recommend security improvements.

---

## 3. Introduction

Modern web applications depend on many software components and third-party libraries. Vulnerabilities in either the application itself or its dependencies can create security risks. DevSecOps integrates security into the software development lifecycle.

### A simplified workflow is:

**Code**
↓
**Build**
↓
**Dependency Analysis (SCA)**
↓
**Run Application**
↓
**Dynamic Security Testing (DAST)**
↓
**Find Vulnerabilities**
↓
**Fix Issues**
↓
**Retest**

---

## 4. What is DAST?

DAST = Dynamic Application Security Testing
DAST tests an application while it is running.
It behaves somewhat like an external attacker by sending HTTP requests to the application and analyzing the responses.

### Example

Web Browser
↓
Running Flask Application
↓
OWASP ZAP
↓
HTTP Requests
↓
Application Responses
↓
Security Findings

DAST can identify issues such as:

- Missing security headers
- Cookie security problems
- Information disclosure
- Input validation issues
- Authentication-related weaknesses
- Other web security vulnerabilities

---

## 5. What is SCA?

SCA = Software Composition Analysis

SCA checks the third-party libraries/dependencies used by an application.

For example:

Flask
Werkzeug
Jinja2

SCA helps determine whether dependencies have known security vulnerabilities or outdated versions.

Difference

| DAST | SCA |
|---|---|
| Tests running application | Tests dependencies |
| Dynamic testing | Dependency analysis |
| Examines HTTP behavior | Examines packages/libraries |
| Finds application-level issues | Finds vulnerable/outdated components |

---

## 6. Objectives

Main objectives

1. Develop a simple web application.
2. Run the application locally.
3. Perform DAST against the running application.
4. Identify security vulnerabilities.
5. Analyze third-party dependencies using SCA.

6. Document the security findings.
7. Recommend remediation techniques.
8. Retest the application after security improvements.

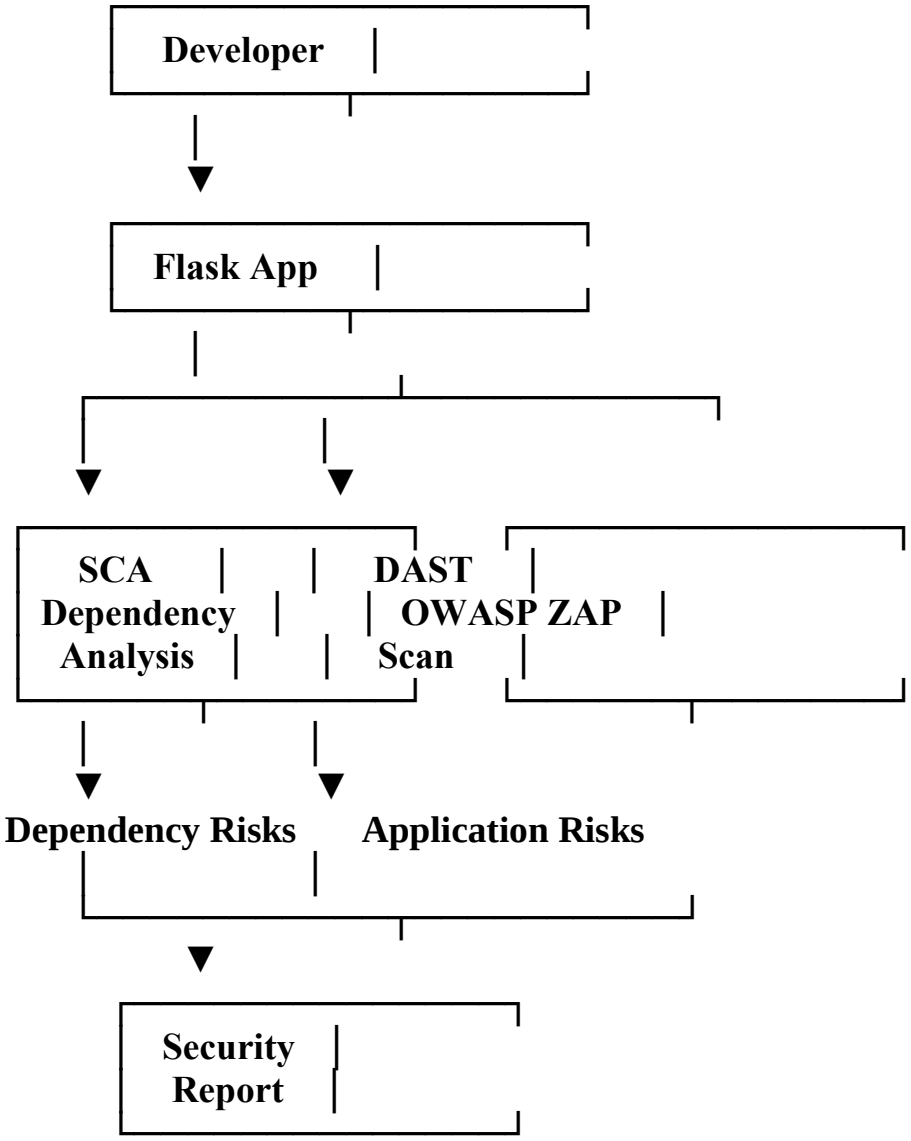
---

## 7. Technologies Required

| Tool | Purpose |
|---|---|
| Python | Programming language |
| Flask | Web application framework |
| OWASP ZAP | DAST scanner |
| pip | Dependency management |
| VS Code | Development |
| Git | Version control |
| GitHub | Repository and submission |

---

## 8. System Architecture



## 9. Project Folder Structure

**Create:**

```
DAST-SCA-Simulation/
│
├── app.py
├── requirements.txt
├── README.md
├── .gitignore
│
├── templates/
│   └── index.html
│
├── static/
│   └── style.css
│
├── screenshots/
│   ├── application.png
│   ├── zap-scan.png
│   └── zap-results.png
│
├── reports/
│   └── security-report.md
│
└── .github/
    └── workflows/
        └── security.yml
```

Your assessment specifically asks students to include editable project/source files and submit the GitHub repository together with the final PDF report.

---

## 10. Flask Application

app.py

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/about")
def about():
    return "<h1>About DAST Simulation</h1>"
```

```
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=False)
```

---

## 11. HTML Page

templates/index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>DAST Simulation</title>
</head>

<body>

    <h1>DAST Simulation Web Application</h1>

    <p>DevSecOps Engineering and Application Security</p>

    <form>
        <label>Username:</label>
        <input type="text" name="username">

        <br><br>

        <label>Password:</label>
        <input type="password" name="password">

        <br><br>

        <button type="submit">Login</button>
    </form>

    <br>

    <a href="/about">About</a>

</body>
</html>
```

---

## 12. Requirements File

requirements.txt

```
Flask
```

Install:

pip install -r requirements.txt
Run:
python app.py
Open:
http://127.0.0.1:5000

---

## 13. DAST Using OWASP ZAP

Now the important practical part.
Step 1
Install OWASP ZAP on your computer.
Step 2
Start your Flask application:
python app.py
Step 3
Open OWASP ZAP.
Step 4
Use the application's local URL:
http://127.0.0.1:5000
Step 5
Run a scan against your own local application.
Step 6
Wait for the scan to complete.
Step 7
Check the Alerts section.
You may see findings categorized by severity such as:
High
Medium
Low
Informational
Don't manually claim a vulnerability unless ZAP actually reports it. Your report should use your actual scan results.

---

## 14. DAST Result Table

After your scan, make a table like this:

| S.No | Vulnerability | Risk | Evidence | Recommendation |
|---|---|---|---|---|
| 1 | Finding from ZAP | High/Medium/Low | Screenshot | Recommended fix |
| 2 | Finding from ZAP | High/Medium/Low | Screenshot | Recommended fix |
| 3 | Finding from ZAP | High/Medium/Low | Screenshot | Recommended fix |

This is better than inventing findings because your assessment evaluates accuracy of the solution. The rubric gives 20% to accuracy and 30% to application of concepts.

---

## 15. SCA

SCA checks the dependencies in:
requirements.txt
For example:
Flask
You can use a dependency vulnerability scanner such as:
pip-audit
Install:
pip install pip-audit
Run:
pip-audit
The tool will check installed Python packages for known vulnerabilities.
Record the result
Example format:

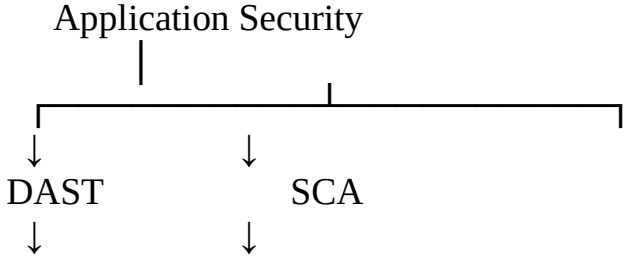
| Package | Version | Vulnerability | Severity | Status |
|---|---|---|---|---|
| Flask | Actual installed version | Actual result | Actual result | Review |
| Package | Actual version | Actual result | Actual result | Review |

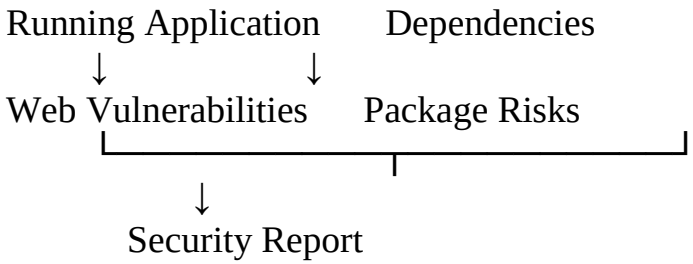
Again, use the actual output from your machine.

---

## 16. DAST vs SCA

DAST
Running Application
↓
OWASP ZAP
↓
HTTP Testing
↓
Security Findings
SCA
requirements.txt
↓
Dependency Scanner
↓
Package Analysis
↓
Known Vulnerabilities
Together:






---

## 17. GitHub Actions / DevSecOps Integration

Create:
.github/workflows/security.yml
Basic structure:

```
name: Security Checks

on:
  push:
  pull_request:

jobs:
  security:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.x"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pip-audit

      - name: Run SCA
        run: pip-audit
```

This demonstrates how security checking can be integrated into a DevSecOps workflow.

---

## 18. Testing Methodology

Phase 1 — Application Development
Create Flask application.
↓
Phase 2 — Dependency Installation

Install required packages.
↓
Phase 3 — SCA
Scan dependencies.
↓
Phase 4 — Application Deployment
Run Flask locally.
↓
Phase 5 — DAST
Scan running application using OWASP ZAP.
↓
Phase 6 — Analysis
Analyze alerts.
↓
Phase 7 — Remediation
Apply appropriate security improvements.
↓
Phase 8 — Retesting
Run the security scan again.

---

## 19. Screenshots You Should Take

For your report, capture:
Screenshot 1: Project folder structure
Screenshot 2: app.py code
Screenshot 3: requirements.txt
Screenshot 4: Flask application running
Screenshot 5: Browser showing 127.0.0.1:5000
Screenshot 6: OWASP ZAP opened
Screenshot 7: ZAP target URL
Screenshot 8: ZAP scan running
Screenshot 9: ZAP alerts/results
Screenshot 10: SCA/pip-audit result
Screenshot 11: GitHub repository
Screenshot 12: GitHub Actions security workflow

---

## 20. Results

The web application was successfully developed and executed locally. Dynamic Application Security Testing was performed against the running application using OWASP ZAP. The identified alerts were analyzed according to their severity and appropriate security recommendations were documented. Software Composition Analysis was also performed on the application's dependencies to identify known dependency-related security risks. The results demonstrate how DAST and SCA can be integrated into a DevSecOps security workflow.

---

## 21. Advantages

- Detects security issues in running applications.
- Helps identify web application weaknesses.
- Provides security findings based on application behavior.
- SCA helps monitor third-party dependencies.
- Supports security integration into DevSecOps.
- Helps developers understand application security.

---

## 22. Limitations

- DAST cannot identify every type of vulnerability.
- Some application functionality may require authentication.
- Scan results may require manual verification.
- SCA depends on available vulnerability databases.
- Security tools can produce false positives.

---

## 23. Future Scope

- Integrate DAST into CI/CD pipelines.
- Add authenticated scanning.
- Add automated security gates.
- Integrate additional SCA tools.
- Generate automated security reports.
- Monitor dependencies continuously.

---

## 24. Conclusion

This project demonstrates the practical implementation of Dynamic Application Security Testing and Software Composition Analysis within a DevSecOps environment. A Flask web application was developed and tested using security tools. DAST was used to analyze the running application, while SCA was used to analyze software dependencies. The combined approach provides better visibility into application and dependency security and demonstrates the importance of integrating security throughout the software development lifecycle.